

STEM Fusion

Introduction to Computational Thinking and Python Programming

THE FOUNDATIONS

Algorithms · Basic Syntax · Variables · Arithmetic · Strings · Input Handling

Instructor: Sarom Leang, PhD
Specialist: Daniela Garcia Galindo
Innovation Hall

Pronouns

Instructor

Sarom Leang, Ph.D.

Pronouns: Teacher / Instructor

Specialist

Daniela Garcia Galindo

Pronouns: Daniela

Schedule



10:00 AM - 10:15 AM

Homeroom



10:15 AM - 11:35 AM

G1



11:40 AM - 12:35 PM

Lunch



12:40 PM - 02:00 PM

G2

Student Expectations



NO FOOD



**NO DRINKS
(on the table)**



Be respectful to
individuals and
property



Be open to
learning



Be open to not
understanding



Be patient with
yourself



Ask questions



Explore



Embrace failure

Course Overview

This course introduces the fundamental building blocks of computational thinking and computer programming using the Python language.

Upon successful completion of this course, students will be able to:

- Improve their problem-solving skills
- Write, read, and execute Python code using basic data types and operators

Course Overview (cont.)

Exploratory Topics

- ~~Session #1 – November 1, 2025~~
 - ~~Entrance Survey~~
 - ~~How Computers Work~~
- ~~Session #2 – December 6, 2025~~
 - ~~Parallel Computing~~
- ~~Session #3 – January 31, 2025~~
 - ~~Generative Artificial Intelligence~~
- Session #4 – March 7, 2026
 - Generative Artificial Intelligence
- Session #5 – April 11, 2026
 - Future of Computing (Quantum)
 - Exit survey

Topics

- Algorithms and Problem Solving
- Debugging and Troubleshooting
- Loops
- Algorithms and Syntax
- Variables and Conditionals
- Variable Arithmetic
- Conditionals (If/Else)
- Compound Conditionals

What Are We Doing Today?



Algorithms



Basic Syntax



Variables



Arithmetic



Strings



Input Handling



Algorithms

The secret sauce behind every app, game, and website you love

What is an Algorithm? 🤔

An algorithm is...

A set of step-by-step instructions that solve a problem or complete a task.

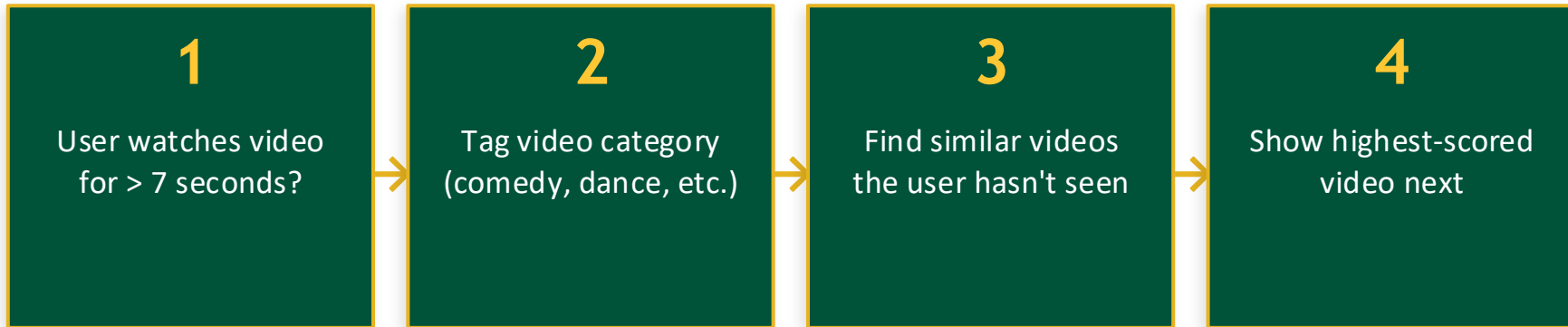
- ✓ Has a clear starting point
- ✓ Each step is precise
- ✓ Eventually finishes
- ✓ Produces a result

You already use algorithms! 🧩

-  Google Maps routing
-  Spotify 'Discover Weekly'
-  TikTok's For You page
-  Google Search ranking
-  Minecraft mob pathfinding

Real World: The TikTok Algorithm

TikTok's recommendation algorithm is just a set of IF / THEN rules running millions of times per second.



 **This loop runs again and again — that's an algorithm!**

Underneath every smart app is someone who wrote out the steps — and that's what you'll learn to do.

Real World: The Spotify Algorithm

Spotify's Discover Weekly is powered by an algorithm that runs every single week for 400 million+ users.

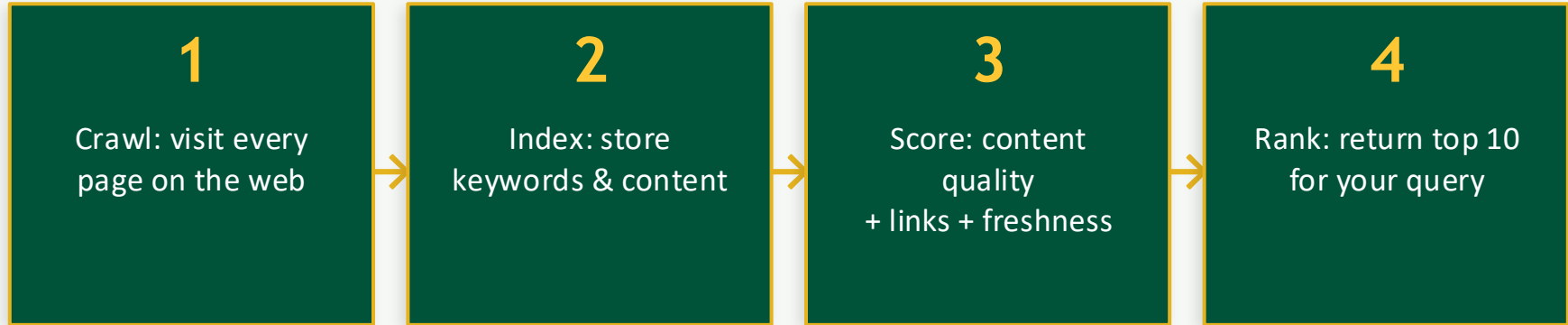


 **This runs 400M times a week — that's the power of a well-designed algorithm!**

Every streaming service, social feed, and search engine runs on the same idea: score → rank → show.

Real World : The Google Search Algorithm

Google crawls over 130 trillion pages and uses an algorithm to rank results in under 0.5 seconds.



PageRank

How many sites link to a page?

Relevance

Do keywords match the query?

Freshness

How recently was it updated?

Pseudocode: Writing Algorithms in Plain English

Make a PB&J sandwich

START

```
GET two slices of bread
SPREAD peanut butter on slice 1
SPREAD jelly on slice 2
PUT slice 2 on top of slice 1
CUT sandwich in half
```

END

Key Pseudocode Rules

- Use plain English — no special symbols needed
- One action per step
- Be specific enough that ANYONE could follow it
- Think about edge cases (what if there's no bread?)
- It doesn't run on a computer — it's a planning tool

⚡ OPTIONAL CHALLENGE — Algorithms



Pen & Paper

Write a pseudocode algorithm for one of the following:



Option A: How to respawn after dying in a video game



Option B: Your morning routine (from alarm → leaving the house)

Make sure your algorithm has:

- A clear START and END
- At least 5 steps
- At least one decision (IF something, THEN...)



Try it in IDLE

```
# In IDLE's interactive shell, try this:  
# (This is pseudocode turned into real code!)
```

```
step = 1  
print('START')  
print('Step', step, ': Wake up')  
step = step + 1  
print('Step', step, ': Brush teeth')  
step = step + 1  
print('Step', step, ': Get dressed')  
print('END')
```

```
# Can you add more steps?
```

Work individually • There's no wrong answer — just think it through!



ANSWERS — Algorithms



Pen & Paper

START

1. Alarm goes off
2. IF too tired → snooze once
3. Get out of bed
4. Brush teeth
5. Get dressed
6. IF hungry → eat breakfast
7. Pack bag
8. Leave the house

END

Key things to look for:

- Clear START and END
- At least 5 specific steps
- At least one IF/THEN decision
- Steps are specific, not vague ('brush teeth' not 'get ready')



IDLE Output

```
>>> step = 1
>>> print('START')
START
>>> print('Step', step, ': Wake up')
Step 1 : Wake up
>>> step = step + 1
>>> print('Step', step, ': Brush teeth')
Step 2 : Brush teeth
>>> step = step + 1
>>> print('Step', step, ': Get dressed')
Step 3 : Get dressed
>>> print('END')
END
```



Basic Syntax

Python's rules — break them and it yells at you 🐍

Syntax: The Grammar of Programming

*"It's WinGARDium LeviOsa,
not LevioSA!"*

— Hermione Granger, debugging since 1991 

Python is exactly the same.
Spell it right, or it won't work.

**Syntax = the set of rules that defines
how a Python program must be written.**

	<code>prnt('Hello')</code>	→	<code>NameError: name 'prnt' is not defined</code>
	<code>print('Hello')</code>	→	<code>Hello</code>

How to Open IDLE on Windows 11

1



Press the Windows key

Bottom-left of keyboard (or click the Start button on the taskbar)

2



Type IDLE in the search bar

The search bar appears automatically when you start typing

3



Click IDLE (Python 3.x)

You'll see it listed under Best Match — the version number may vary

4



The Shell window opens

You'll see >>> — this is where you type and run Python code

5



Test it: type `print('Hello')` then Enter

If you see Hello printed below, IDLE is working correctly!

Python's Golden Rules

1

Case-sensitive

`print()` $\neq$ `Print()` $\neq$ `PRINT()`

2

Indentation matters

Use 4 spaces (or 1 tab) consistently — Python uses this to understand structure

3

No curly braces

Unlike Java/C++, Python uses indentation instead of { }

4

Strings need quotes

Use 'single' or "double" — both work, just be consistent

5

Comments start with

This is a comment — Python ignores it.
Use them to explain your code.

6

One statement per line

Generally, keep each action on its own line for readability

Your First Python Program: Hello, World!

```
# My first Python program  
print('Hello, World!')  
print('My name is Python 🐍')  
print('Nice to meet you!')
```

OUTPUT:

```
Hello, World!  
My name is Python 🐍  
Nice to meet you!
```

starts a comment → Python ignores this line

print() displays text on screen

Text in quotes = a 'string' (more soon!)

⚡ OPTIONAL CHALLENGE — Basic Syntax



Pen & Paper

Find the 5 syntax errors in this pseudocode-turned-Python. Circle each error and write what the fix would be:

```
Print('Hello')
prnt('My age is', 16)
print(How are you?)
PRINT('Today is Monday')
print('2 + 2 =', 2+2
```

Bonus: What does each corrected line actually output?



Try it in IDLE

```
# Open IDLE and type these lines.
# See which ones work and which crash.
# Fix them all!
```

```
Print('Hello')
prnt('My age is', 16)
print(How are you?)
PRINT('Today is Monday')
print('2 + 2 =', 2+2
```

```
# Hint: there are exactly 5 errors.
# Read the error messages carefully!
```



ANSWERS — Basic Syntax



Pen & Paper

Find the 5 syntax errors in this pseudocode-turned-Python. Circle each error and write what the fix would be:

```
Print('Hello')
prnt('My age is', 16)
print(How are you?)
PRINT('Today is Monday')
print('2 + 2 =', 2+2
```

Bonus: What does each corrected line actually output?



IDLE Output

```
>>> print('Hello')
Hello

>>> print('My age is', 16)
My age is 16

>>> print('How are you?')
How are you?

>>> print('Today is Monday')
Today is Monday

>>> print('2 + 2 =', 2+2)
2 + 2 = 4

# Error messages students will see:
# NameError: name 'Print' is not defined
# SyntaxError: unexpected EOF
# Always read the message!
```



Variables

Labeled boxes where your code stores stuff

Variables: Labeled Boxes in Memory



← variable name (the label)

← value stored inside

```
player_name = 'Jake'  
score = 0  
lives = 3  
is_playing = True
```


Variable Names & Data Types

Naming Rules

- ✓ letters, numbers, underscores
- ✓ `player_name` `score_1` `my_var`
- ✗ Cannot start with a number (`1score`)
- ✗ No spaces (`player name`)
- ✗ No special chars (`score!`)
- ⚡ Use snake_case by convention
- ⚡ Be descriptive: `age` not `a`

Common Data Types

int	<code>score = 100</code>	<i>whole numbers</i>
------------	--------------------------	----------------------

float	<code>gpa = 3.8</code>	<i>decimal numbers</i>
--------------	------------------------	------------------------

str	<code>name = "Jake"</code>	<i>text (strings)</i>
------------	----------------------------	-----------------------

bool	<code>alive = True</code>	<i>True or False</i>
-------------	---------------------------	----------------------

Variables in Action: Student Report Card

student_name	math_grade	gpa	honor_roll
"Alex"	95	3.8	True

← variable name (the label)

← value stored inside

```
# Report card
student_name = 'Alex'
math_grade = 95
gpa = 3.8
honor_roll = True

print(student_name)
print('GPA:', gpa)
```

Variables Can Change

Reassigning Variables

```
score = 0
print(score)    # 0

score = 50
print(score)    # 50

score = score + 10
print(score)    # 60
```

The variable **score** is updated each time.
The old value is replaced — not saved.

Checking Types with type()

```
# What type is this variable?
name = 'Alex'
print(type(name))
# <class 'str'>

grade = 95
print(type(grade))
# <class 'int'>

gpa = 3.8
print(type(gpa))
# <class 'float'>
```

Useful when debugging — if your code behaves unexpectedly, check the type!

⚡ OPTIONAL CHALLENGE — Variables



Pen & Paper

Create a 'character profile' using variables. Your character can be from any game, movie, or show you like.

Your profile must include:

- A name (string)
- An age or level (integer)
- A power level or score (float)
- Whether they are the hero or not (boolean)

Write the pseudocode assignment statements (no Python syntax needed yet).



Try it in IDLE

```
# Create your character profile in IDLE!
# Replace the values with your own character.

character_name = 'Spider-Man'
age = 17
power_level = 9.5
is_hero = True

# Now print each one:
print('Name:', character_name)
print('Age:', age)
print('Power Level:', power_level)
print('Hero?', is_hero)

# Try changing the values and re-running!
```



ANSWERS — Variables



Pen & Paper

Create a 'character profile' using variables. Your character can be from any game, movie, or show you like.

Your profile must include:

- A name (string)
- An age or level (integer)
- A power level or score (float)
- Whether they are the hero or not (boolean)

Write the pseudocode assignment statements (no Python syntax needed yet).

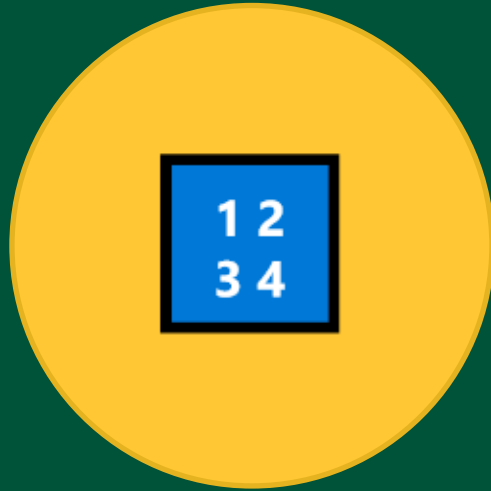


IDLE Output

```
>>> character_name = 'Spider-Man'
>>> age = 17
>>> power_level = 9.5
>>> is_hero = True

>>> print('Name:', character_name)
Name: Spider-Man
>>> print('Age:', age)
Age: 17
>>> print('Power Level:', power_level)
Power Level: 9.5
>>> print('Hero?', is_hero)
Hero? True

# type() check:
>>> type(character_name)
<class 'str'>
>>> type(power_level)
<class 'float'>
```



Arithmetic

Python can do math. Way faster than you. 

Python Arithmetic Operators $+$ $-$ $\times$ $\div$

$+$

Addition

$5 + 3 \rightarrow 8$

$-$

Subtraction

$10 - 4 \rightarrow 6$

$*$

Multiplication

$3 * 7 \rightarrow 21$

$/$

Division

$9 / 2 \rightarrow 4.5$

$//$

Floor Div

$9 // 2 \rightarrow 4$

$\%$

Modulus

$9 \% 2 \rightarrow 1$

$**$

Exponent

$2 ** 8 \rightarrow 256$

Order of Operations & Real-World Math

Python follows PEMDAS 

```
# 2 + 3 * 4 = ?  
print(2 + 3 * 4)    # 14  
print((2+3) * 4)    # 20  
print(2 ** 3 + 1)   # 9
```

1. Parentheses
2. Exponents
3. Multiply / Divide
4. Add / Subtract

 Real-World Example: Temperature Converter

```
# Convert Celsius to Fahrenheit  
# Formula: F = (C × 9/5) + 32  
  
celsius = 100  
fahrenheit = (celsius * 9/5) + 32  
print(fahrenheit)  # 212.0
```

*Mars average temp = -63°C
What is that in Fahrenheit?*

PEMDAS Practice: Evaluate Each Expression



Without running Python, what does each expression evaluate to?

1 `print( 3 + 4 * 2 )`

Watch out — which operation runs first?

2 `print( (10 - 2) ** 2 / 4 )`

Work from the inside out

3 `print( 15 % 4 + 2 * 3 )`

% has the same priority as * and /



Work it out on paper first — check your answer on the next slide

PEMDAS Solutions — Step by Step

Apply PEMDAS: Parentheses → Exponents → Multiply/Divide → Add/Subtract

1 $3 + 4 * 2$

M $4 * 2 \rightarrow 8$



A $3 + 8 \rightarrow 11$

2 $(10 - 2) ** 2 / 4$

P $(10 - 2) \rightarrow 8$



E $8 ** 2 \rightarrow 64$



D $64 / 4 \rightarrow 16.0$

3 $15 \% 4 + 2 * 3$

M $15 \% 4 \rightarrow 3$
(remainder)



M $2 * 3 \rightarrow 6$



A $3 + 6 \rightarrow 9$

⚡ OPTIONAL CHALLENGE — Arithmetic



Pen & Paper



Fantasy Stats Challenge:

A basketball player scored 28 points per game over 82 games. They also earned a \$3.2 million bonus for every championship won (they won 2).

On paper, calculate:

1. Total season points
2. Total bonus money
3. Average points if they played 5 more games scoring 20 pts each

Show your work with the operators you'd use (+, -, *, /).



Try it in IDLE

```
# Solve it in Python!
points_per_game = 28
games = 82
championships = 2
bonus_per_title = 3_200_000

total_points = points_per_game * games
total_bonus = championships *
bonus_per_title

# Bonus: avg with extra games
extra_games = 5
extra_points = 20
new_avg = (total_points + extra_games *
extra_points) / (games + extra_games)

print('Season points:', total_points)
print('Bonus earned: $', total_bonus)
print('New average:', new_avg)
```



ANSWERS — Arithmetic



Pen & Paper



Fantasy Stats Challenge:

A basketball player scored 28 points per game over 82 games. They also earned a \$3.2 million bonus for every championship won (they won 2).

On paper, calculate:

1. Total season points
2. Total bonus money
3. Average points if they played 5 more games scoring 20 pts each

Show your work with the operators you'd use (+, -, *, /).



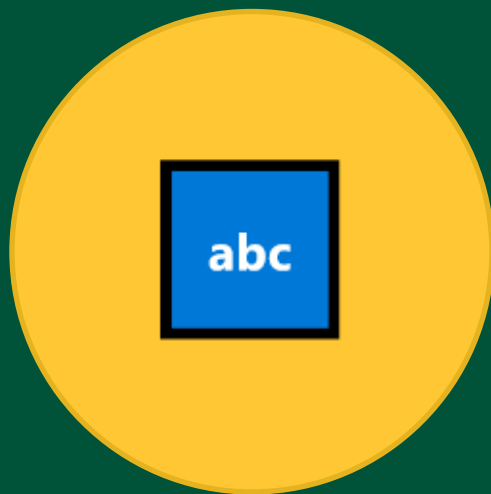
IDLE Output

```
>>> total_points = 28 * 82
>>> print('Season points:', total_points)
Season points: 2296
```

```
>>> total_bonus = 2 * 3_200_000
>>> print('Bonus: $', total_bonus)
Bonus: $ 6400000
```

```
>>> new_avg = (2296 + 5*20) / (82+5)
>>> print('New average:', new_avg)
New average: 27.540229885057472
```

```
# Round to 2 decimal places:
>>> print(round(new_avg, 2))
27.54
```



Strings

Working with words, sentences, and text data

Strings: Text in Your Program

A string is any sequence of characters inside quotes — letters, numbers, spaces, emojis, symbols. Strings are data.

Creating Strings

```
song = 'Anti-Hero'  
artist = "Taylor Swift"  
year = '2022'  
lyric = "It's me, hi!"
```

String Operations

```
# Concatenation (joining)  
full = song + ' by ' + artist  
print(full)  
# Anti-Hero by Taylor Swift  
  
# Length  
print(len(song)) # 9
```

Handy String Tricks

`len()`

`len("Taylor") → 6`

Count characters

`.upper()`

`"hello".upper() → HELLO`

All uppercase

`.lower()`

`"HELLO".lower() → hello`

All lowercase

`[index]`

`"Swift"[0] → S`

Get one character (starts at 0)

`* N`

`"Na" * 3 → NaNaNa`

Repeat a string

`in`

`"Tay" in "Taylor" → True`

Check if substring exists

More String Tricks

`.replace()`

```
"Hi Bob".replace("Bob", "Alex")  
→ Hi Alex
```

Swap text out

`.strip()`

```
"  hello  ".strip()  
→ hello
```

Remove whitespace

`.split()`

```
"a,b,c".split(",")  
→ ['a', 'b', 'c']
```

Break into a list

`.find()`

```
"Taylor".find("a")  
→ 1
```

Find index of substring

`[start:end]`

```
"Taylor"[0:3]  
→ Tay
```

Slice a portion

`f-strings`

```
name = "Alex"  
f"Hello, {name}!"  
→ Hello, Alex!
```

Embed variables in text

⚡ OPTIONAL CHALLENGE — Strings



Pen & Paper

Build a Spotify-style track info display using only string operations:

Given:

```
artist = 'Kendrick Lamar'
```

```
song = 'Not Like Us'
```

```
plays = 'billion'
```

Using pen & paper, write the Python lines that would print:

```
TRACK: NOT LIKE US
```

```
Artist: Kendrick Lamar
```

```
This song has 1 billion plays!
```

```
Song title has 11 characters.
```

Hint: You'll need `upper()`, `len()`, `+`, and `print()`.



Try it in IDLE

```
# Now try it in IDLE!
```

```
artist = 'Kendrick Lamar'
```

```
song = 'Not Like Us'
```

```
plays = 'billion'
```

```
# Build the output:
```

```
print('TRACK: ' + song.upper())
```

```
print('Artist: ' + artist)
```

```
print('This song has 1 ' + plays + ' plays!')
```

```
print('Song title has', len(song), 'characters.')
```

```
# Bonus challenge:
```

```
# Can you print the artist's
```

```
# first name only?
```

Work individually • There's no wrong answer — just think it through!



ANSWERS — Strings



Pen & Paper

Build a Spotify-style track info display using only string operations:

Given:

```
artist = 'Kendrick Lamar'
```

```
song = 'Not Like Us'
```

```
plays = 'billion'
```

Using pen & paper, write the Python lines that would print:

```
TRACK: NOT LIKE US
```

```
Artist: Kendrick Lamar
```

```
This song has 1 billion plays!
```

```
Song title has 11 characters.
```

Hint: You'll need `upper()`, `len()`, `+`, and `print()`.



IDLE Output

```
>>> artist = 'Kendrick Lamar'
```

```
>>> song = 'Not Like Us'
```

```
>>> plays = 'billion'
```

```
>>> print('TRACK: ' + song.upper())
```

```
TRACK: NOT LIKE US
```

```
>>> print('Artist: ' + artist)
```

```
Artist: Kendrick Lamar
```

```
>>> print('This song has 1 ' + plays + ' plays!')
```

```
This song has 1 billion plays!
```

```
>>> print('Song title has', len(song), 'characters.')
```

```
Song title has 11 characters.
```

```
# Bonus:
```

```
>>> artist.split()[0]
```

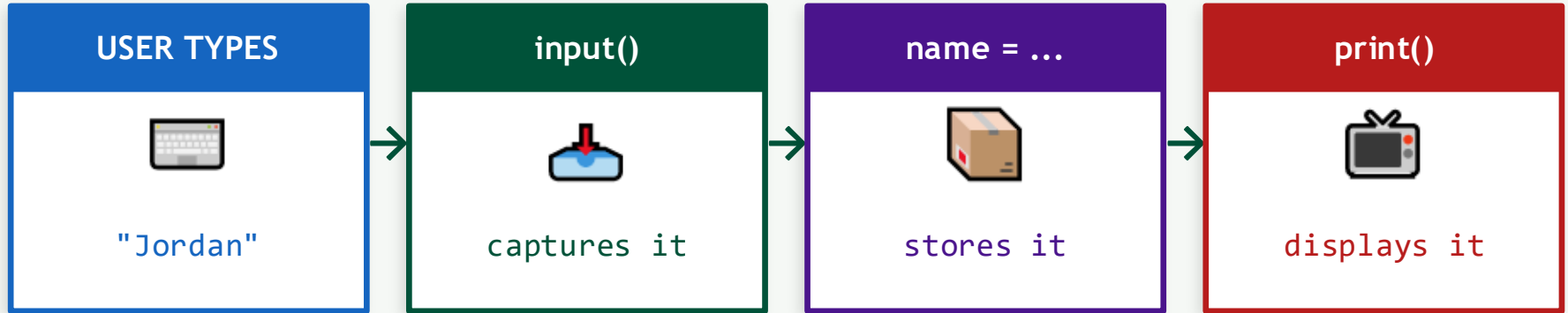
```
'Kendrick'
```



Input Handling

Making your programs actually talk back

The input() Function



```
# Ask the user for their name
name = input("What is your name? ")
print("Nice to meet you,", name)
print("Welcome to Python,", name + "!")
```

OUTPUT:

```
What is your name? Jordan
Nice to meet you, Jordan
Welcome to Python, Jordan!
```

Important: input() Always Returns a String

 input() ALWAYS returns a string — even if the user types a number!

 This crashes:

```
age = input("Your age? ")  
print(age + 1)  
# TypeError: can only  
# concatenate str to str
```

 This works:

```
age = int(input("Your age? "))  
print(age + 1)  
# If user types 16:  
# Output: 17
```

int() converts a string to an integer. float() converts to a decimal number.

⚡ OPTIONAL CHALLENGE — Input Handling



Pen & Paper

Build a 'Would You Rather?' program on paper.

Your program should:

1. Ask the user: 'Would you rather fly or be invisible?'
2. Store their answer in a variable called choice
3. Print: 'Interesting! You chose: [their answer]'
4. Ask their name and print a personalized goodbye

Write the 4 lines of pseudocode/Python needed.



Try it in IDLE

```
# Build it in IDLE!
choice = input('Would you rather fly or be invisible? ')
print('Interesting! You chose:', choice)

name = input('What is your name? ')
print('See you later,', name + '! Stay awesome.')

# BONUS: Ask for their age and
# calculate how old they'll be in 10 years!
age = int(input('How old are you? '))
print('In 10 years you will be', age + 10)
```



ANSWERS — Input Handling



Pen & Paper

Build a 'Would You Rather?' program on paper.

Your program should:

1. Ask the user: 'Would you rather fly or be invisible?'
2. Store their answer in a variable called choice
3. Print: 'Interesting! You chose: [their answer]'
4. Ask their name and print a personalized goodbye

Write the 4 lines of pseudocode/Python needed.



IDLE Output

```
>>> choice = input('Fly or invisible? ')
Fly or invisible? fly

>>> print('Interesting! You chose:', choice)
Interesting! You chose: fly

>>> name = input('What is your name? ')
What is your name? Jordan

>>> print('See you later,', name+'! Stay awesome.')
See you later, Jordan! Stay awesome.

# Bonus:
>>> age = int(input('How old are you? '))
How old are you? 15
>>> print('In 10 years you will be', age + 10)
In 10 years you will be 25
```



Complete!

Look what you can do now:



Think algorithmically



Write valid Python
syntax



Create & use
variables



Do math in Python



Work with strings



Build interactive
programs

Next up: Boolean Logic → If Statements → Loops 

Questions? Review challenges? Open coding time!

Innovation Computer Issues

Front Screen